

Хранение в K8s

Volumes, PV, PVC, StorageClass

Кирилл Касаткин
DevOps-инженер, Renue

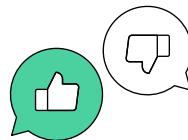


Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

- ☒ + если меня видно и слышно
- ☐ — если нет

Рекомендации

→ Если смотрите с компьютера

- Используйте браузеры **Google Chrome** или **Microsoft Edge**
- Если есть проблемы с изображением или звуком, обновите страницу — **F5**

→ Если смотрите с мобильного телефона или планшета

- Перейдите с мобильного интернет-соединения на **Wi-Fi**
- Если есть проблемы с изображением или звуком, перезапустите приложение **МТС-Линк** на телефоне

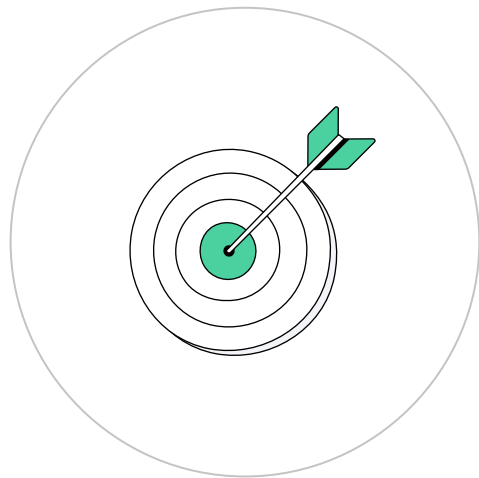
Кирилл Касаткин

DevOps-инженер, Renue



Цели занятия

- Узнать, что такое Volume
 - где находятся файлы контейнера
 - как использовать Shared Volumes для обмена данными между контейнерами
 - типы Volume и их назначение
- Изучить, что такое PersistentVolume (PV), PersistentVolumeClaim (PVC) и как они работают
- Познакомиться с концепцией StorageClass и его ролью в автоматизации

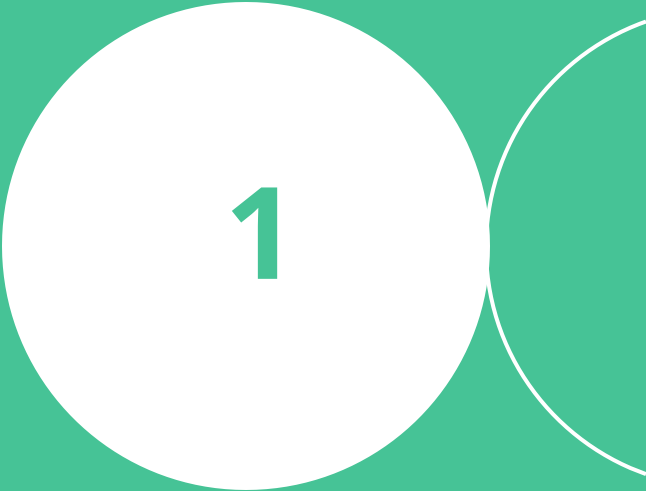


План занятия

- 1 Файловая система контейнера
- 2 Volume
- 3 Shared volumes
- 4 Типы volume
- 5 Типы volume, продолжение
- 6 PersistentVolume
- 7 PersistentVolumeClaim
- 8 StorageClass
- 9 Итоги занятия



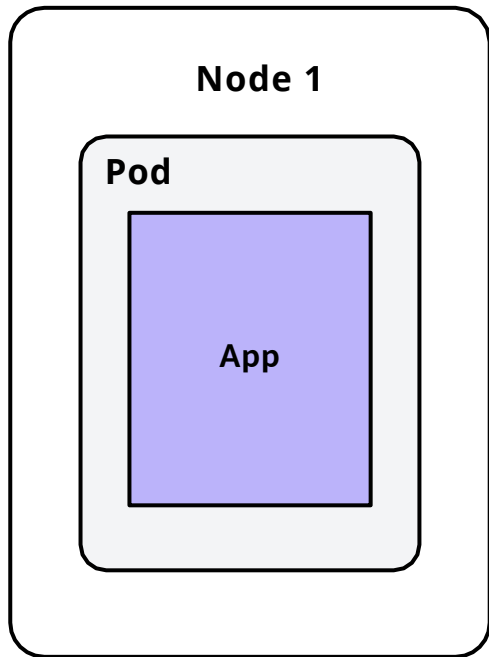
Файловая система контейнера



1

Файловая система контейнера в Pod

Файловая система контейнера **эфемерна**



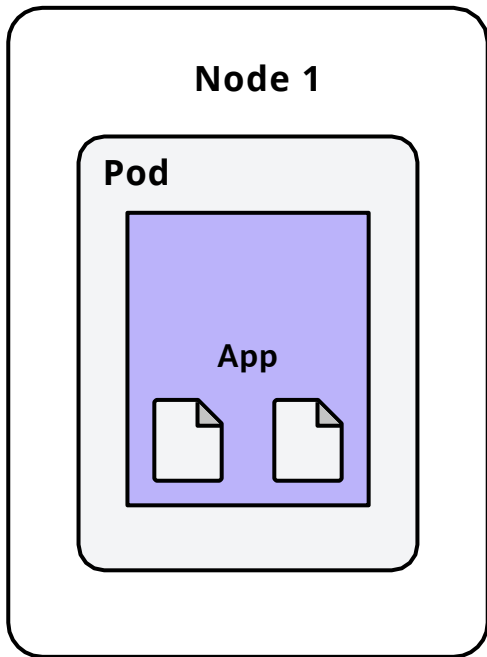
*Эфемерна — временная, не сохраняется после завершения работы контейнера



Источник: архив автора

Файловая система контейнера в Pod

Файловая система контейнера **эфемерна**

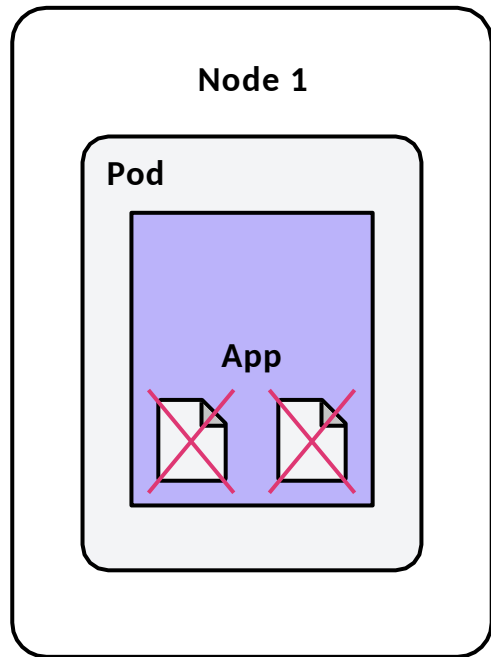


→ Файлы в контейнере существуют, пока существует контейнер



Файловая система контейнера в Pod

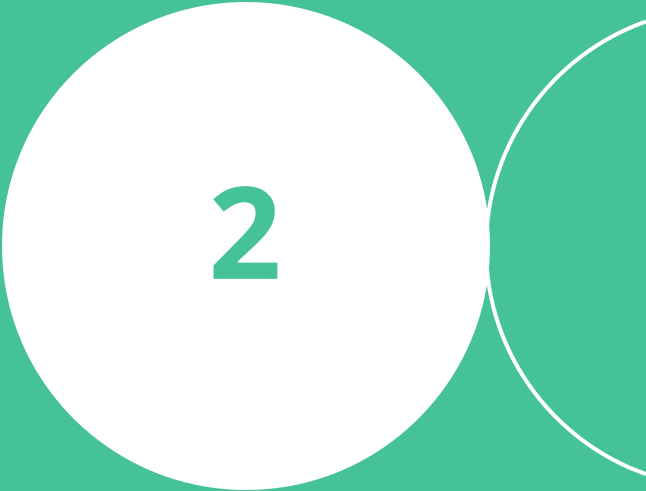
Файловая система контейнера **эфемерна**



- Файлы в контейнере существуют, пока существует контейнер
- С удалением или перезапуском контейнера все изменения файловой системы исчезнут



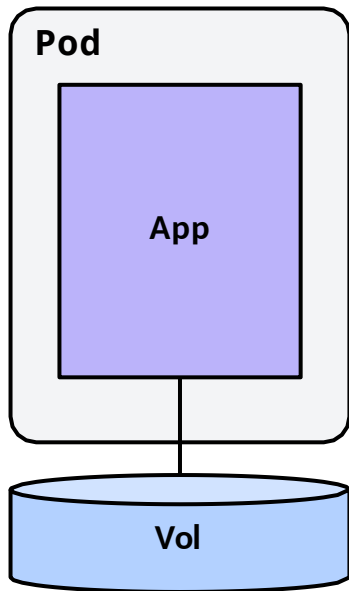
Volume



2

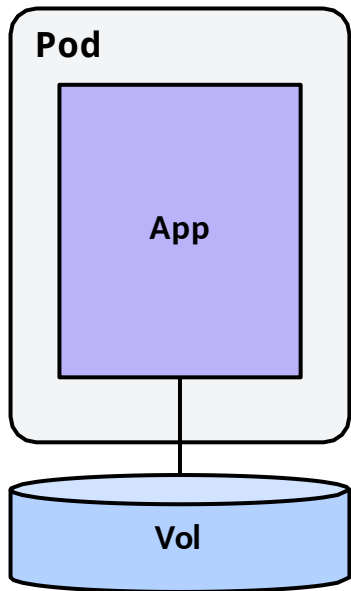
Volume

Volume (том) — директория, хранящаяся за пределами пода



Volume

Volume (том) — директория, хранящаяся за пределами пода.

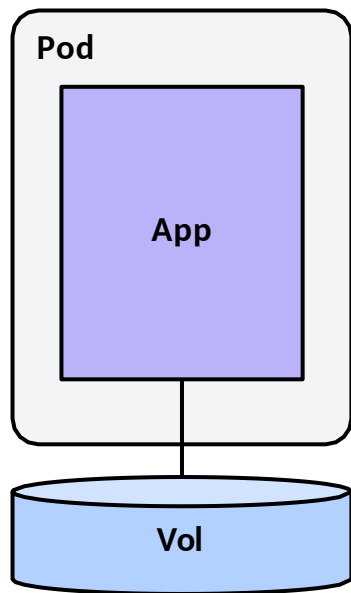


- Том объявляется на уровне пода и монтируется внутри нужного контейнера



Volume

Volume (том) — директория, хранящаяся за пределами пода

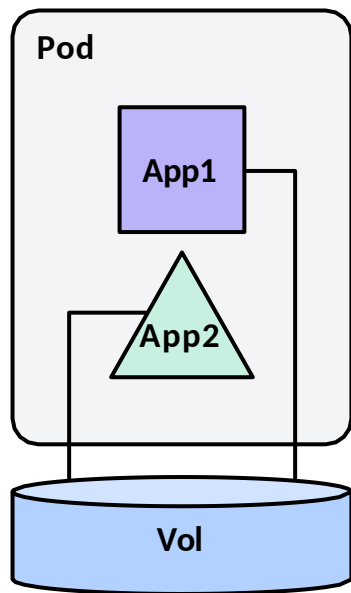


- Том объявляется на уровне пода и монтируется внутри нужного контейнера
- Путь монтирования объявляется на уровне отдельного контейнера



Volume

Volume (том) — директория, хранящаяся за пределами пода



- Том объявляется на уровне пода и монтируется внутри нужного контейнера
- Путь монтирования объявляется на уровне отдельного контейнера
- Если в поде несколько контейнеров, то этот путь для каждого из них может различаться



Пример конфигурации

Пример конфигураций Pod с подключением volume

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-pod
spec:
  containers:
    - name: app1
      image: busybox
      command: ['sh', '-c', 'echo Success! > /output/success.txt']
      volumeMounts:
        - name: vol
          mountPath: /output
  volumes:
    - name: vol
      hostPath:
        path: /var/data
```



Пример конфигурации

Пример конфигураций Pod с подключением volume

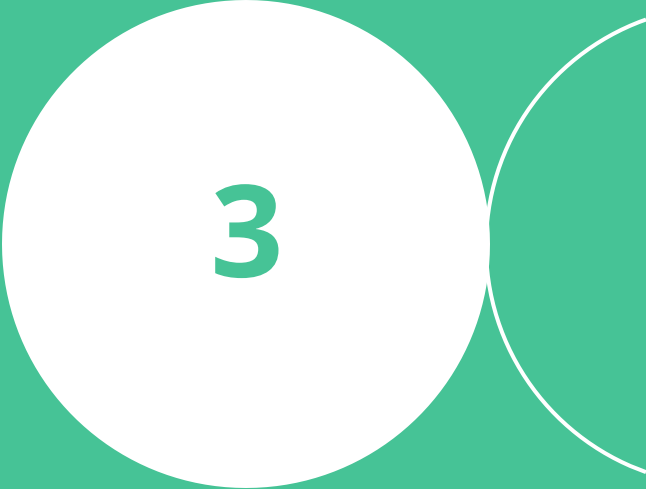
```
apiVersion: v1
kind: Pod
metadata:
  name: volume-pod
spec:
  containers:
    - name: app1
      image: busybox
      command: ['sh', '-c', 'echo Success! > /output/success.txt']
      volumeMounts:
        - name: vol
          mountPath: /output
  volumes:
    - name: vol
      hostPath:
        path: /var/data
```





Ваши вопросы?

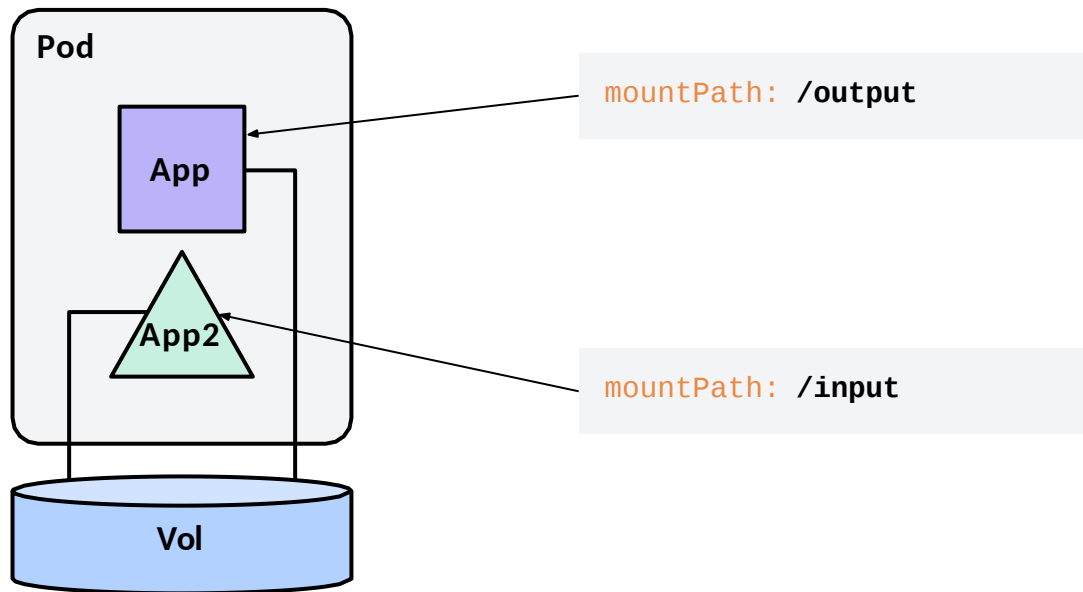
Shared volumes



3

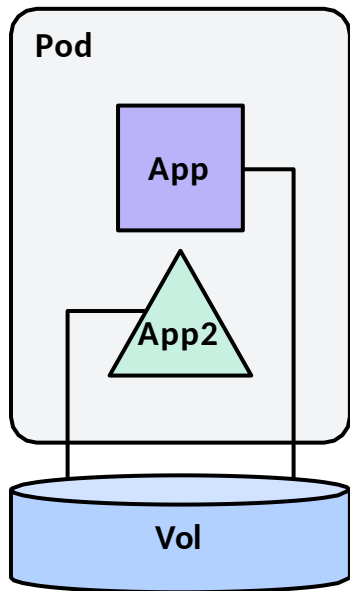
Shared volume между контейнерами в поде

Volume можно использовать как общую папку между контейнерами внутри пода



Shared volume между контейнерами в поде

Volume можно использовать как общую папку между контейнерами внутри пода



```
apiVersion: v1
kind: Pod
metadata:
  name: volume-pod
spec:
  containers:
    - name: app1
      image: busybox
      command: ['sh', '-c', 'echo Success! > /output/success.txt']
      volumeMounts:
        - name: vol
          mountPath: /output
    - name: app2
      image: busybox
      command: ['sh', '-c', 'while true; do cat /input/success.txt']
      volumeMounts:
        - name: vol
          mountPath: /input
  volumes:
    - name: vol
      hostPath:
        path: /var/data
```



Типы volume

4

Типы volume

Существует множество типов volume. Условно их можно разделить на:

- **локальные**, расположенные непосредственно на ноде, где находится под
- остальные, определяемые с помощью Persistent Volume



Типы volume

Существует множество типов volume. Условно их можно разделить на:

- **локальные**, расположенные непосредственно на ноде, где находится под
- остальные, определяемые с помощью Persistent Volume (будет рассмотрено в следующей лекции)

Наиболее часто встречаемые **локальные типы**:

- emptyDir
- hostPath



Тип `volume emptyDir`

Создаётся и удаляется вместе с подом, но при сбое и его дальнейшем самовосстановлении (перезапуске) данные из этого `volume` не удаляются.

При этом, если под был удалён администратором и затем автоматически перезапущен, данные из `volume` исчезнут

```
volumes:  
- name: my-volume  
  emptyDir: {}
```



Тип volume emptyDir

Создаётся и удаляется вместе с подом, но при сбое и его дальнейшем самовосстановлении (перезапуске) данные из этого volume не удаляются.

При этом, если под был удалён администратором и затем автоматически перезапущен, данные из volume исчезнут

Обычно используется для:

- данных, которые необходимо сохранить при сбое в работе контейнера: например, кеш-файлов
- обмена файлами между несколькими контейнерами в поде

```
volumes:  
- name: my-volume  
  emptyDir: {}
```



Тип volume hostPath

Используется, когда необходимо предоставить доступ к локальной файловой системе ноды. Определяет путь, где будут храниться данные

Файлы на хостах доступны для записи только подам, запущенным с правами пользователя root

```
volumes:  
- name: my-volume  
  hostPath:  
    path: /var/data
```



Закрепляем

Вопрос: могут ли контейнеры в одном поде обмениваться файлами? Если могут, то как?



Закрепляем

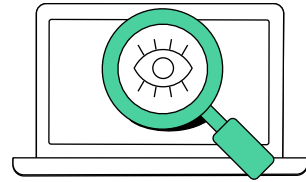
Вопрос: могут ли контейнеры в одном поде обмениваться файлами? Если могут, то как?

Ответ: Могут. Через Shared Volumes. Под объявляет локальный volume, каждый контейнер монтирует его в свою файловую систему



Демонстрация работы

Работа с volume в Pod



Типы Volume

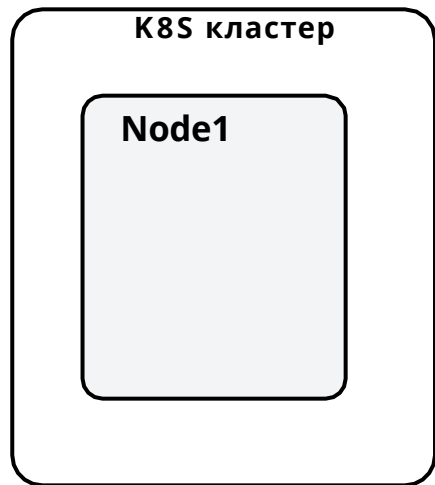
продолжение



5

Типы Volume

Volume можно рассматривать как абстрактный ресурс кластера, как CPU или RAM, который используется для хранения данных (Storage)

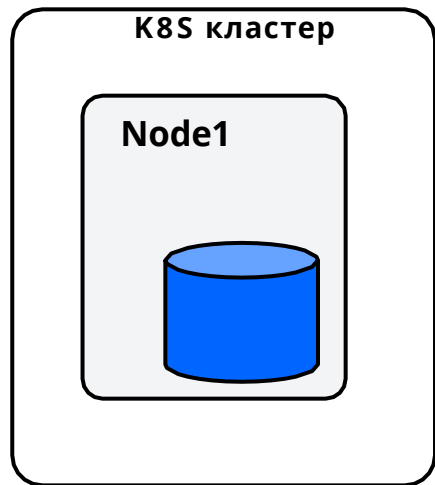


Физически может быть:



Типы Volume

Volume можно рассматривать как абстрактный ресурс кластера, как CPU или RAM, который используется для хранения данных (Storage)



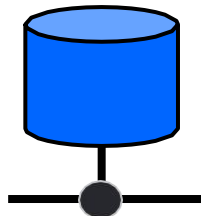
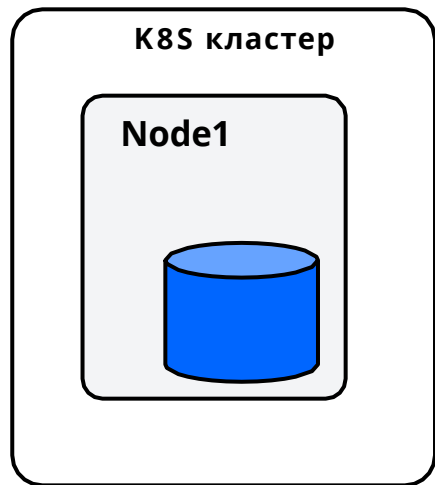
Физически может быть:

- Локальное хранение (пример — HDD)



Типы Volume

Volume можно рассматривать как абстрактный ресурс кластера, как CPU или RAM, который используется для хранения данных (Storage)



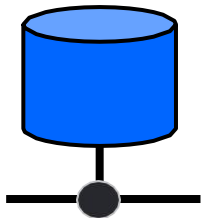
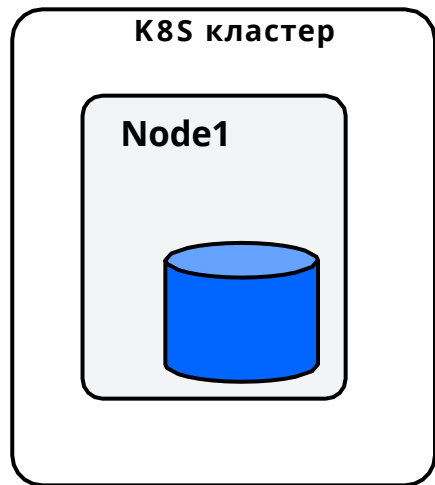
Физически может быть:

- Локальное хранение (пример — HDD)
- Сетевое хранение (пример — NFS)



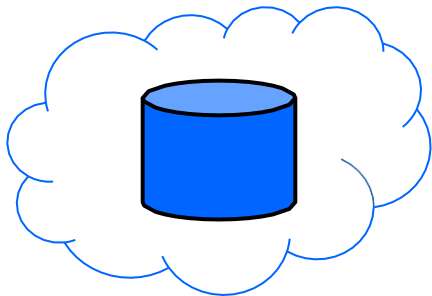
Типы Volume

Volume можно рассматривать как абстрактный ресурс кластера, как CPU или RAM, который используется для хранения данных (Storage)

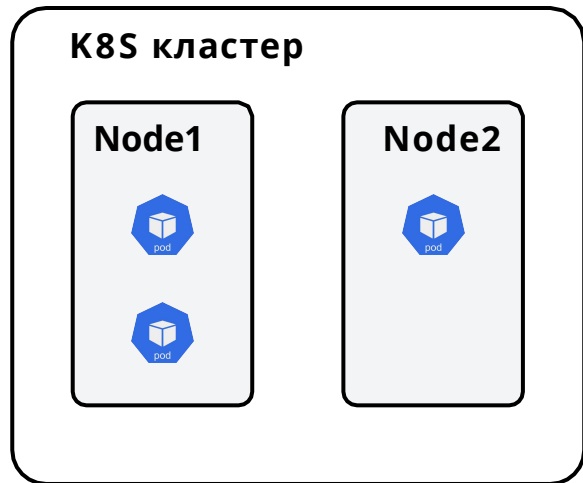


Физически может быть:

- Локальное хранение (пример — HDD)
- Сетевое хранение (пример — NFS)
- Облачное хранение (пример — S3)



Типы Volume

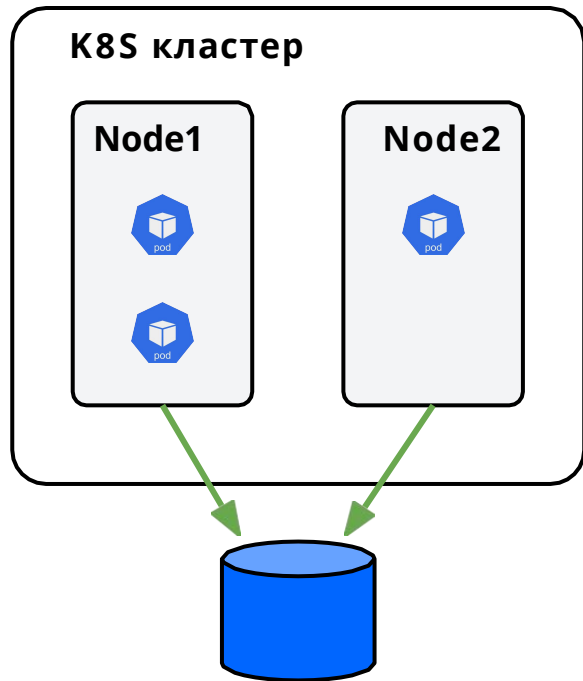


При этом Volume:

- Не зависит от подов и нод



Типы Volume

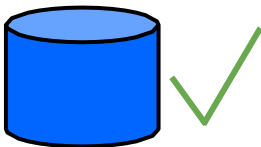
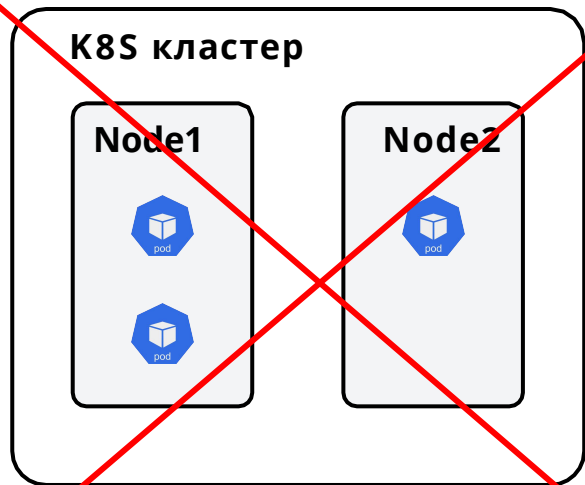


При этом Volume:

- Не зависит от подов и нод
- Должен быть доступен из всех нод



Типы Volume

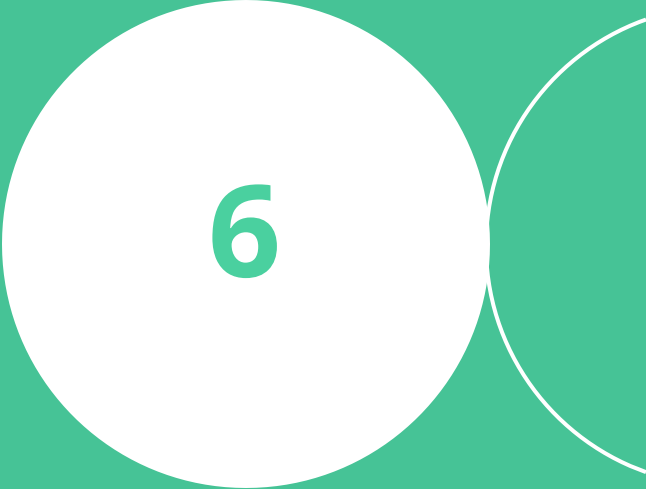


При этом Volume:

- Не зависит от подов и нод
- Должен быть доступен из всех нод
- Выжить после падения кластера (не зависит от кластера)



PersistentVolume



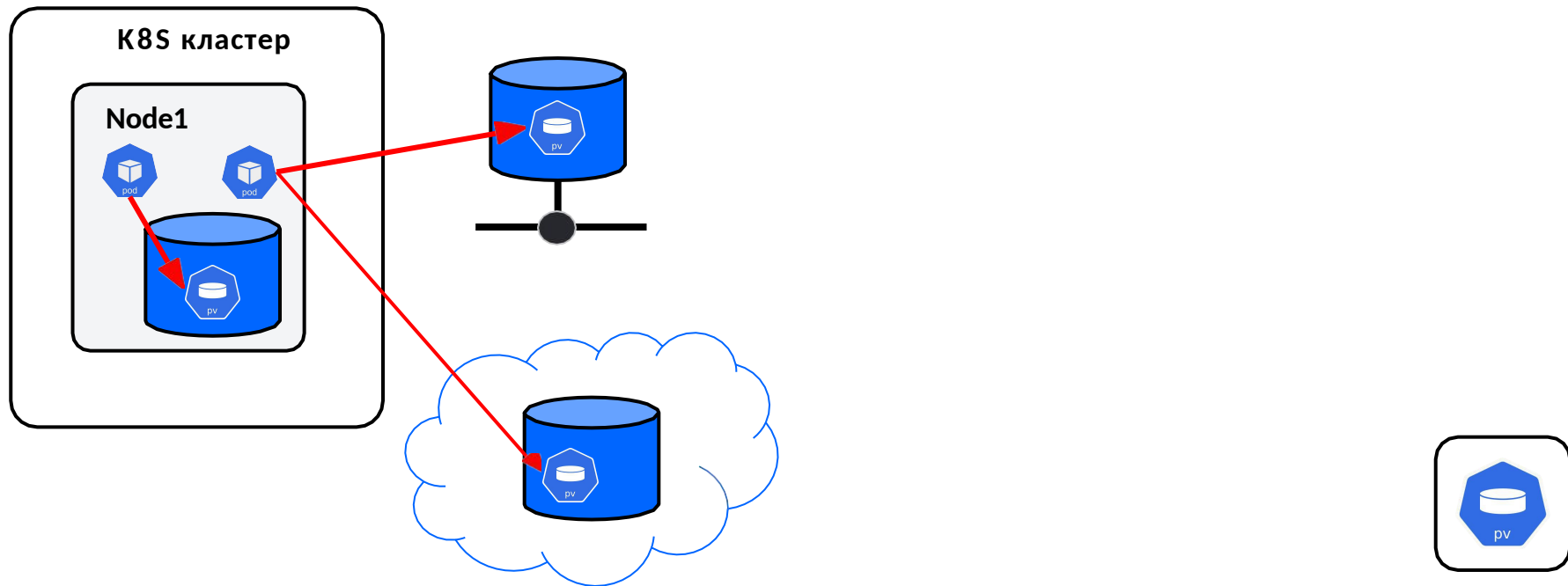
6



**PersistentVolume (PV) —
объект k8s, который
позволяет обращаться
с хранилищем как
к абстрактным ресурсом**

PersistentVolume (PV) — постоянный том

Pod может использовать абстрактный ресурс хранения, а PV выступает как интерфейс или плагин к типу хранения



PersistentVolume (PV) — постоянный том

PV, как и другие объекты K8S, определяется манифестом в формате YAML

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: google-volume
  labels:
    failure-domain.beta.kubernetes.io/zone: us-central1-a us-central1-b
spec:
  capacity:
    storage: 400Gi
  accessModes:
    - ReadWriteOnce
  gcePersistentDisk:
    pdName: my-data-disk
    fsType: ext4
```

Пример конфигурации
Google Cloud



PersistentVolume (PV) — постоянный том

PV, как и другие объекты K8S, определяется манифестом в формате YAML

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: local-volume
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-storage
  local:
    path: /mnt/disk/ssd1
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - example-node
```

Пример конфигурации локального диска



PersistentVolume (PV) — постоянный том

AccessMode (режимы доступа к томам)

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/pv1
```

- **ReadWriteOnce** — (RWO) volume может быть примонтирован в режиме чтения и записи только к одной ноде
- **ReadOnlyMany** — (ROX) volume может быть примонтирован в режиме только чтения ко множеству нод
- **ReadWriteMany** — (RWX) volume может быть примонтирован в режиме чтения и записи ко множеству нод
- **ReadWriteOncePod** — (RWOP) volume может быть примонтирован в режиме чтения и записи только к одному поду



PersistentVolume (PV) — постоянный том

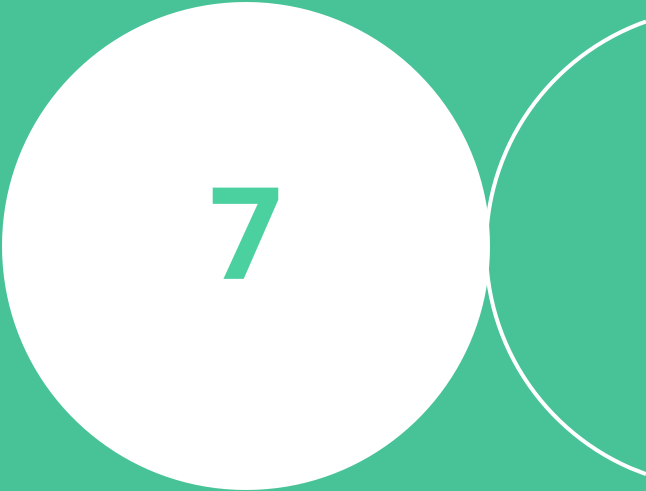
ReclaimPolicy — определяет как будут использованы ресурсы после удаления PV

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/pv1
  persistentVolumeReclaimPolicy: Retain
```

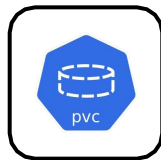
- **Retain** — после удаления PV ресурсы из внешних провайдеров автоматически не удаляются
- **Delete** — после удаления PV ресурсы из внешних провайдеров автоматически удаляются (работает только в облачных Storage)
- **Recycle** — [rm-rf] автоматически удаляет ресурсы (устаревшее)



PersistentVolumeClaim



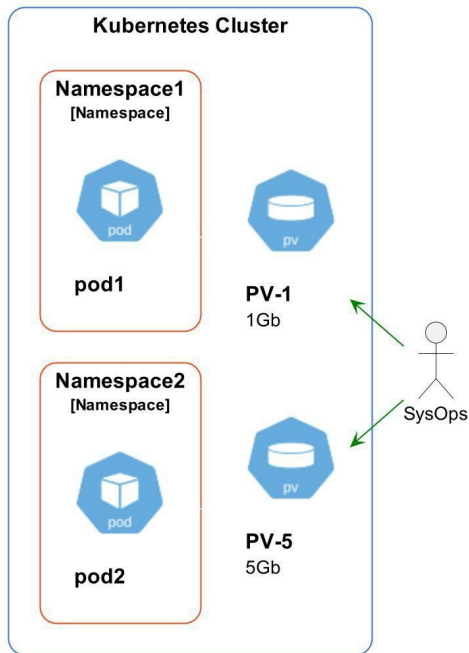
7



PersistentVolumeClaim (PVC) — запрос (заявка) на выделение тома

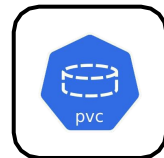
PersistentVolumeClaim (PVC) — заявка на том

PV не определяется в настройках Pod'a и Namespace



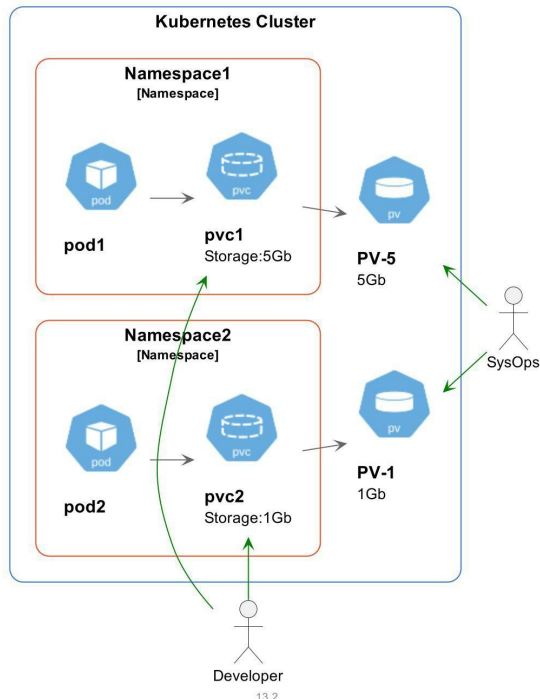
Настройкой хранения занимается Системный администратор:

- Настройка типа хранения (NFS, Cloud, Local)
- Настройка **PV**



PersistentVolumeClaim (PVC) — заявка на том

PV не определяется в настройках Pod'a и Namespace

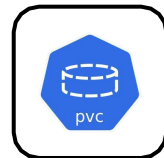


Настройкой хранения занимается Системный администратор:

- Настройка типа хранения (NFS, Cloud, Local)
- Настройка **PV**

Настройкой приложения и запроса на хранение занимается Разработчик:

- Настройка Pod'a, Namespace
- Настройка **PVC**

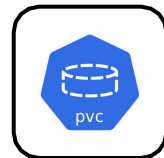


PersistentVolumeClaim (PVC) — заявка на том

PVC определяется в настройках Pod'а и должен быть в том же Namespace

```
apiVersion: v1
kind: Pod
metadata:
  name: app1
  namespace: web
spec:
  containers:
    - name: frontend
      image: nginx
      volumeMounts:
        - mountPath: "/var/www/html"
          name: vol
  volumes:
    - name: vol
      persistentVolumeClaim:
        claimName: pvc-vol
```

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-vol
  namespace: web
spec:
  storageClassName: manual
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

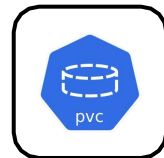


PersistentVolumeClaim (PVC) — заявка на том

Связка (bound) PVC — PV происходит при совпадении характеристик

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-vol
  namespace: web
spec:
  storageClassName: manual
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

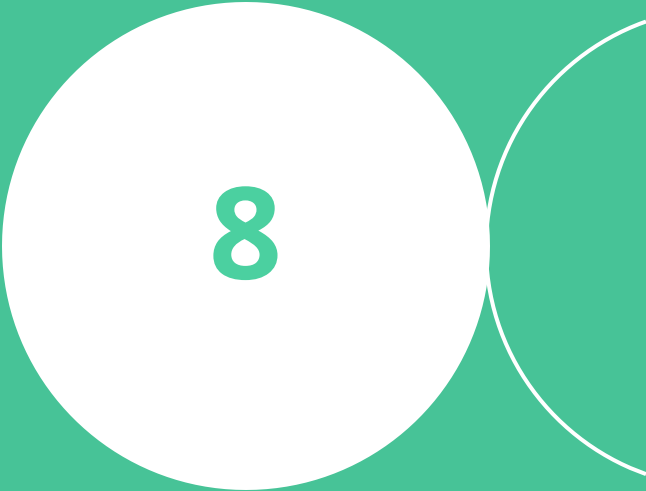
```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv1
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/pv1
```





Ваши вопросы?

StorageClass



8



StorageClass — объект K8s, который описывает класс хранилища

StorageClass — класс хранилища

SC позволяет администратору определить тип хранилищ, доступных ресурсам кластера k8s

- name - имя класса
- provisioner - драйвер хранилища
- parameters - параметры



StorageClass — класс хранилища

SC позволяет администратору определить тип хранилищ, доступных ресурсам кластера k8s

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: name:
  aws-ebs
provisioner: kubernetes.io/aws-ebs
parameters:
  type: io1
  iopsPerGB: "10"
  fsType: ext4
```

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
```

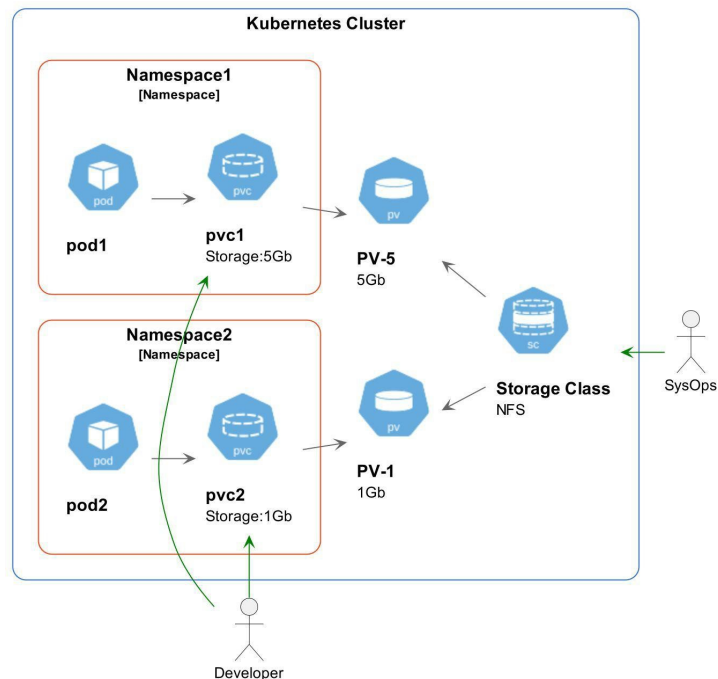
- name - имя класса
- provisioner - драйвер хранилища
- parameters - параметры

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: my-nfs
provisioner: example.com/external-nfs
parameters:
  server: nfs-server.example.com
  path: /share
  readOnly: "false"
```



StorageClass — класс хранилища

SC позволяет администратору создать динамическое создание PV. При этом тома выделяются автоматически и эти тома соответствуют тем параметрам, которые требуются



Для динамического выделения томов необходимо подключить один или больше **provisioner**

Бывают:

- internal — kubernetes.io
- external — внешний



Источник: архив автора

StorageClass — класс хранилища

Provisioner SC создает PV автоматически при создании PVC ровно тех характеристик, которые описаны в PVC

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-vol
  namespace: web
spec:
  storageClassName: my-nfs
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: my-nfs
provisioner: example.com/external-nfs
parameters:
  server: nfs-server.example.com
  path: /share
  readOnly: "false"
```



Закрепляем

Вопрос: Зачем нужен PersistentVolume и PersistentVolumeClaim?



Закрепляем

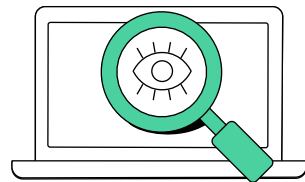
Вопрос: Зачем нужен PersistentVolume и PersistentVolumeClaim?

Ответ: PV — представление хранилища в кластере, не зависит от пода.
PVC — для выделения поду хранилища из существующего PV



Демонстрация работы

PV, PVC, SC



Итоги занятия

9

Итоги занятия

- Узнали, что такое Volume
 - где находятся файлы контейнера
 - как использовать Shared Volumes для обмена данными между контейнерами
 - типы Volume и их назначение
- Изучили, что такое PersistentVolume (PV), PersistentVolumeClaim (PVC) и как они работают
- Познакомились с концепцией StorageClass и его ролью в автоматизации



Хранение в K8s

Volumes, PV, PVC, StorageClass

Кирилл Касаткин
DevOps-инженер, Renue

